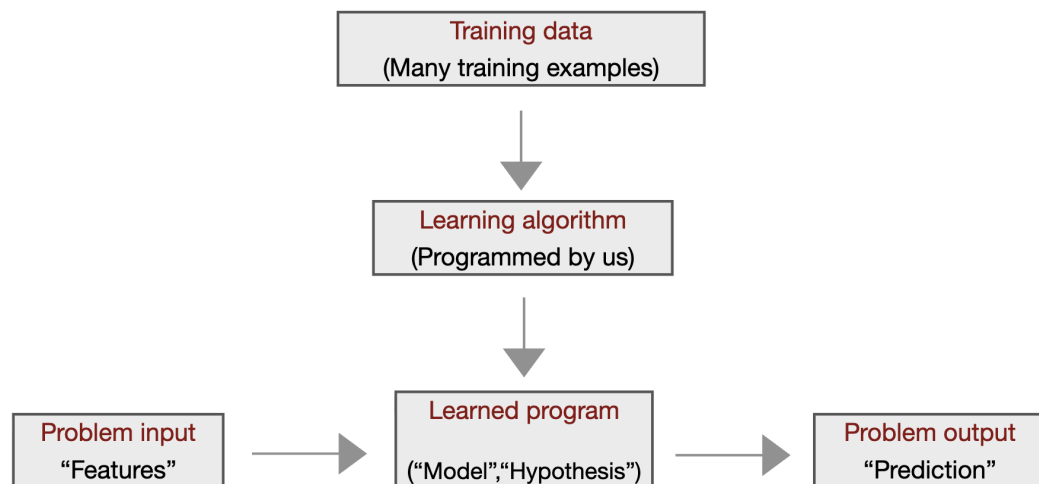


Supervised Learning

Supervised Learning



Supervised Learning Definitions

A **hypothesis** or **model** or **prediction** function is a function $\mathbf{f}: \mathcal{X} \rightarrow \mathcal{Y}$

Spam classification: $\mathcal{X}$ is an email, $\mathcal{Y}$ is “spam” or “not spam”
 $\mathcal{X} = \{0, \dots, 255\}^{10,000}$ and $\mathcal{Y} = \{0, 1\}$

Stop sign detection: $\mathcal{X}$ is an image, $\mathcal{Y}$ are the coordinates of the sign
 $\mathcal{X} = [0, 1]^{244 \times 244 \times 3}$ and $\mathcal{Y} = \{0, 1, \dots, 244\}^8$

House price prediction: $\mathcal{X}$ are characteristics of a house, $\mathcal{Y}$ is the price
 $\mathcal{X} = \mathbb{R}^d$ and $\mathcal{Y} = \mathbb{R}$

If $\mathcal{Y}$ is continuous, then we call it a **regression** problem.

If $\mathcal{Y}$ is discrete, then we call it a **classification** problem (*binary* or *multi-class*).

hypothesis=model=predictor

goal is to use the training set to find (= learn) a good model
we will want to use the model f on new data, not the training set

input vector [element1, element2, ...] $x^{(i)}$ $\rightarrow$ output vector [...] $y^{(i)}$

****Design Decisions****

Which predictors are possible?: Hypothesis class

How **good** is a predictor?: Loss/Cost function

How do we compute the best predictor?: Optimization algorithm

Supervised Learning - Probabilistic Algorithms

- **Linear Regression**
- **Logistic Regression**
- **Softmax Regression**
- **GLMs**

Linear Regression (Least Squares)

Hypothesis Class: "Linear Function" (Linear in terms of parameters)

$$y^{(i)} = f(x^{(i)}) + \epsilon^{(i)}$$

$$= \underbrace{w_0 h_0(x^{(i)})}_{\text{feature 1}} + \underbrace{w_1 h_1(x^{(i)})}_{\text{feature 2}} + \dots + \underbrace{w_D h_D(x^{(i)})}_{\text{feature D+1}} + \epsilon^{(i)}$$

* $h(x)$ is a 'feature extractor'. Each $h(x^{(i)})$ 'extracts' information from the input vector $x^{(i)}$ and turns it into a scalar value, which each becomes the 'feature'.

* ϵ is noise — $\mathbb{E}[\epsilon] = 0$

* $h_0(x) = 1$

* $h(x)$ could be anything, e.g. x^p , $\sin(2\pi x/3)$, $\log(x)$

*matrix notation

$$h(x) = [h_0(x^{(i)}), h_1(x^{(i)}), \dots, h_D(x^{(i)})] \quad // \text{'feature extractor'}$$

$$w = [w_0, w_1, \dots, w_D] \quad // \text{'weight vector'}$$

$$f_w(x) = w \cdot h(x) = w^T h(x) = h(x)^T w$$

To do computation all at once, we can put all data points $(x^{(1)}, x^{(2)}, \dots, x^{(n)})$ into a matrix, in other words, stack up $h(x)^T$ s, like below:

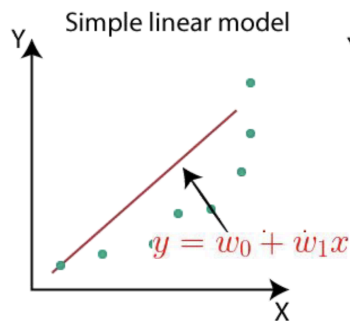
$$\begin{array}{l} \mathbf{Y} = \mathbf{H}\mathbf{w} + \mathbf{e} \\ \mathbf{Y} : n \times 1 \\ \mathbf{H} : n \times d \\ \mathbf{w} : d \times 1 \\ \mathbf{e} : n \times 1 \end{array} \quad \begin{array}{c} \left(\begin{array}{c} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{array} \right) = \left(\begin{array}{cccc} h_1(x^{(1)}) & h_2(x^{(1)}) & \dots & h_d(x^{(1)}) \\ h_1(x^{(2)}) & h_2(x^{(2)}) & \dots & h_d(x^{(2)}) \\ \vdots & \vdots & \ddots & \vdots \\ h_1(x^{(n)}) & h_2(x^{(n)}) & \dots & h_d(x^{(n)}) \end{array} \right) \left(\begin{array}{c} w_1 \\ w_2 \\ \vdots \\ w_d \end{array} \right) + \left(\begin{array}{c} e^{(1)} \\ e^{(2)} \\ \vdots \\ e^{(n)} \end{array} \right)$$

Y
H
w
e

→ w_0 should be included

*Below, we assume $x^{(i)}$ is a scalar value (only one element in the vector).

If we choose to use only $h_0(x)$ as 1 and $h_1(x)$ as x ,
we get a straight line which we often refer to 'linear regression'



If we choose to use x^p 's as $h(x)$,
we get a polynomial line, which we often refer to 'polynomial regression'.

Model:

$$y^{(i)} = w_0 + w_1 x^{(i)} + w_2 (x^{(i)})^2 + \dots + w_p (x^{(i)})^p + \epsilon^{(i)}$$

treat as different "features"

feature 1 = 1 (constant)

feature 2 = x

feature 3 = x^2

...

feature $p+1 = x^p$

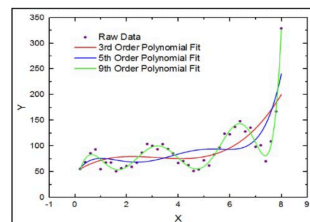
parameter 1 = w_0

parameter 2 = w_1

parameter 3 = w_2

...

parameter $p+1 = w_p$



Loss(Cost) Function : Residual Sum of Squares

$$Loss(x, y, w) = (f_w(x) - y)^2$$

$$TrainLoss(w) = \frac{1}{|D_{train}|} \sum_{(x,y) \in D_{train}} Loss(x, y, w)$$

$$* \sum_{(x,y) \in D_{train}} Loss(x, y, w) = \text{RSS(Residual Sum of Squares)}$$

* $f_w(x)$ is linear in terms of w && Loss function uses RSS $\rightarrow$ RSS(w) is convex

*Where does squared loss come from? - Probabilistic Explanation

Assume Gaussian Noise model, with mean 0, then Maximum Likelihood Estimation (MLE) of the parameters (w) tells us the solution is squared loss

$$y^{(i)} = w^T h(x^{(i)}) + \epsilon^{(i)}, \text{ where } \epsilon \sim N(0, \sigma^2) \text{ and IID } // y^{(i)} | x^{(i)}; w \sim N(w^T h(x^{(i)}), \sigma^2)$$

$$p(y^{(i)} | x^{(i)}; w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - w^T h(x^{(i)}))^2}{2\sigma^2}\right)$$

// Notation: semicolon in $p(y|x^{**}; w)$ — w is a parameter, not a random variable, so we don't condition on it (that would be $p(y|x, w)$)

$$\text{Likelihood} = \text{probability of the data as a function of } w: L(w) = \prod_{i=1}^n p(y^{(i)} | x^{(i)}; w)$$

// product only because of IID

Maximum likelihood estimation: pick w that makes the observed data most probable

$$\operatorname{argmax}_w L(w) = \operatorname{argmax}_w \log L(w) = \operatorname{argmax}_w \log \left[\prod_{i=1}^n p(y^{(i)} | x^{(i)}; w) \right]$$

$$\operatorname{argmax}_w \sum_{i=1}^n \log p(y^{(i)} | x^{(i)}; w) = \operatorname{argmax}_w \sum_{i=1}^n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \log \frac{(y^{(i)} - w^T h(x^{(i)}))^2}{2\sigma^2}$$

$$= \operatorname{argmin}_w (y^{(i)} - w^T h(x^{(i)}))^2$$

$\rightarrow$ maximizing $\ell(w) \Leftrightarrow$ minimizing RSS(w). Least squares = MLE under Gaussian noise

*Two caveats:

- The answer doesn't depend on σ^2 — same w^* even if σ^2 unknown (this fact gets reused in Ch 3, GLMs)
- Gaussian assumption is sufficient, not necessary — other assumptions also justify least squares

Optimization Algorithm: Gradient descent (Least mean squared algorithm)

How do we find the “best fit” line?

-> w^* that minimize the loss(RSS) over the training data.

How do we minimize the RSS?

-> Apply gradient descent to RSS

*LMS means applying gradient descent to a (mean) squared loss form. RSS is ‘squared loss’ not ‘mean squared loss’ but it doesn’t matter.

1. Batch Gradient Descent

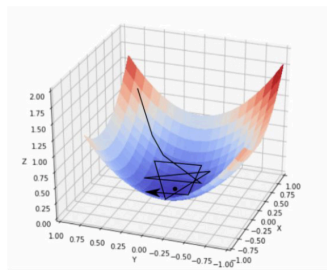
each step sums gradients over the entire training set

Goal: $\min_w \text{TrainLoss}(w)$



Definition: gradient

The gradient $\nabla_w \text{TrainLoss}(w)$ is the direction that increases the training loss the most.



Algorithm: gradient descent

Initialize $w = [0, \dots, 0]$

For $t = 1, \dots, T$: epochs

$$w \leftarrow w - \underbrace{\eta}_{\text{step size}} \underbrace{\nabla_w \text{TrainLoss}(w)}_{\text{gradient}}$$

$$w^{(t+1)} \leftarrow w^{(t)} - \underbrace{\eta \nabla \text{RSS}(w^{(t)})}_{-2H^T(y-Hw)}$$

(using ∇RSS instead of $\nabla \text{TrainLoss}$ (just $\times n$))

*For step size - two strategies: constant / shrinking ($\eta(t) = \frac{\alpha}{t}$ or $\eta(t) = \frac{\alpha}{\sqrt{t}}$)

*Step size is a hyperparameter: a parameter that is not part of the model, something that you as a designer will try to set or try to discover, but don't typically learn it in the same way as you learn the model parameters

*Stop when $\left| \frac{d\text{TrainLoss}}{dw} \right| < \epsilon$ or when a maximum number of iterations is done (for convex functions, convergence happens when gradient = 0, but in practice, that doesn't happen often)

*If we differentiate on a single data point by a single parameter w_j :

$$\frac{\partial \text{Loss}}{\partial w_j} = 2(f_w(x) - y)h_j(x) = 2 \times (\text{error}) \times (\text{value of feature } j)$$

→ Shows adjusting each parameter in proportion to how wrong the prediction was, and how much that feature contributed to it.

2. Stochastic Gradient Descent

updates immediately using a single example

```
// Hyperparameter: learning rate  $\alpha$ , number of total iteration  $n_{iter}$ 
```

```
Initialize  $\theta$  randomly
```

```
For  $i = 1$  to  $n_{iter}$ :
```

```
    sample  $j$  uniformly from  $\{1, \dots, n\}$  and update  $\theta$  by  $\theta := \theta - \alpha \nabla_{\theta} J^{(j)}(\theta)$ 
```

Why Does SGD Work At All? (Intuition) - For “ascent” (maximizing): gradient is the “best” direction but any direction that goes “up” would be useful

$$\nabla \text{TrainLoss}(w) = \left[\frac{\partial \text{TrainLoss}(w)}{\partial w_1}, \frac{\partial \text{TrainLoss}(w)}{\partial w_2}, \dots, \frac{\partial \text{TrainLoss}(w)}{\partial w_n} \right]$$

Gradient descent

$$\frac{\partial \text{TrainLoss}(w)}{\partial w_j} = \frac{\partial \sum_{i=1}^n \text{Loss}(x_i, y_i, w)}{\partial w_j}$$

Stochastic Gradient descent

$$\frac{\partial \text{TrainLoss}(w)}{\partial w_j} \approx \frac{\partial \text{Loss}(x_i, y_i, w)}{\partial w_j}$$

SG Update for L2-Regularized Objective

$$\text{Total derivative} = \sum_{i=1}^N \frac{\partial \ell_i(\mathbf{w})}{\partial \mathbf{w}_j} - 2 \lambda \mathbf{w}_j \quad \boxed{\text{partial}[j]}$$

Stochastic gradient update

Each time, pick different data point i

$$\text{Total derivative} \approx \frac{\partial \ell_i(\mathbf{w})}{\partial \mathbf{w}_j} - \frac{2}{N} \lambda \mathbf{w}_j \quad \boxed{\text{partial}[j]}$$

Each data point contributes $1/N$ to regularization

-> for regularized objectives, divide the objective by n

*Batch vs SGD:

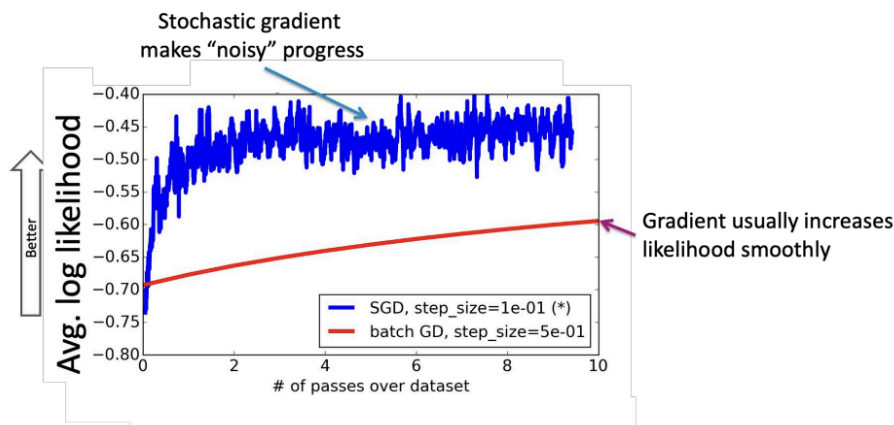
Batch

- every step follows the exact gradient direction, but slow
- cost per step $O(n)$, expensive for large n

SGD

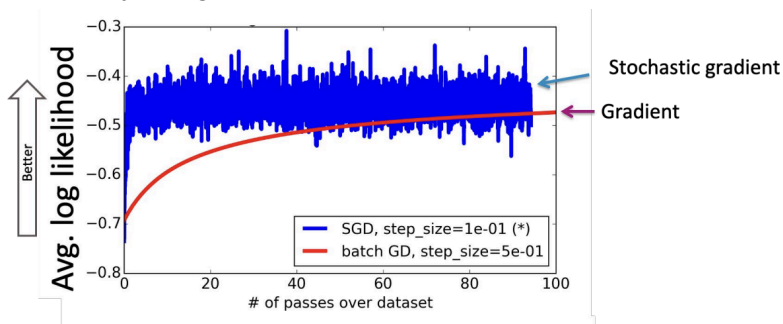
- cost per step $O(1)$, makes progress right away, gets "close" to the minimum much faster

*SGD's gradient is a noisy estimate → it achieves higher likelihood sooner, and does very well when far away from the optimum. However it does poorly (and oscillates) when close to the optimum (never converges)



- Fix 1: Shrinking step size ($\eta(t) = \alpha/t$ above) — as $\eta \rightarrow 0$, convergence to the global minimum is guaranteed
- Fix 2: Use average of the oscillating parameters

Eventually, full gradient catches up.



In practice SGD is almost always preferred for large n .

Mini-batch GD = compromise between the two (update on batches of B examples)

```
// Hyperparameter: learning rate  $\alpha$ , Batch size  $\beta$ , number of total iteration  $n_{iter}$ 
Initialize  $\theta$  randomly

For  $i = 1$  to  $n_{iter}$ :
    sample  $B$  examples  $j_1, \dots, j_B$  (without replacement) uniformly from  $\{1, \dots, n\}$ 
    and update  $\theta$  by  $\theta := \theta - \frac{\alpha}{\beta} \sum_{k=1}^B \nabla_{\theta} J^{(j_k)}(\theta)$ 
```

Optimization Algorithm: The Normal Equation

Closed-form solution(As opposed to 'iterative')

$$\begin{aligned} \text{RSS}(w) &= \sum_{i=1}^n (y^{(i)} - h^T(x^{(i)})w)^2 \\ &= (y - \mathbf{H}w)^T (y - \mathbf{H}w) \end{aligned}$$

$$\begin{aligned} \text{RSS}(w) &= \sum_{i=1}^n (y^{(i)} - (h(x^{(i)}))^T w)^2 = (y - Hw)^T (y - Hw) \\ &= (y^T - w^T H^T)(y - Hw) = y^T y - w^T H^T y - y^T Hw + w^T H^T Hw = y^T y - 2y^T Hw + w^T H^T Hw \end{aligned}$$

$$\nabla_w \text{RSS}(w) = \frac{\partial}{\partial w} \text{RSS}(w) = \frac{\partial}{\partial w} (y^T y - 2y^T Hw + w^T H^T Hw) = -2H^T y + 2H^T Hw = 0$$

Closed form solution(Normal Equation): $H^T Hw = 2H^T y$

- For a closed-form solution to exist, the loss function has to be quadratic in w . It makes $\nabla_w \text{RSS}(w)$ affine in w , so setting it to zero yields a solvable linear system — hence the closed-form solution, the 'normal equation,' exists.

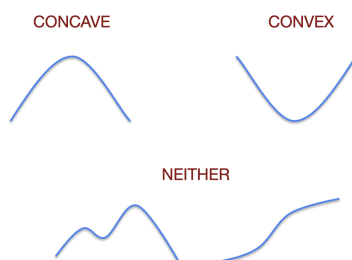
$$w^* = (H^T H)^{-1} H^T y$$

*In order to **compute** the closed-form solution, $H^T H$ should have an inverse. In other words, loss function should be strictly convex. There is only one parameter that minimizes the loss function. For that to happen, the columns of H must be linearly independent.

If it doesn't have an inverse(such as when there are more features than the data - $n < d$ makes columns of H linearly dependent), there will be infinitely many ideal parameters that minimize the loss function.

- In this case we can use other options to just display all of these infinite numbers of solutions, or use the pseudo-inverse to pick one(the minimum norm solution - it is more inefficient as d gets bigger, since the cost of calculating the inverse exceeds the cost of doing gradient descent.) This minimum norm solution tends to generalize well. Moreover, GD with zero initialization automatically converges to this minimum norm solution without any explicit regularization penalty(implicit regularization effect)

*If the loss function is convex, we are ensured the gradient=0 point is the global minimum.



Locally Weighted Linear Regression (LWR)

Motivation: with plain linear regression, performance hinges on choosing good features (underfit line vs. overfit 5th-degree polynomial). LWR makes the choice of features less critical, given enough training data. Instead of fitting one global line, it re-fits a line at every query point using only nearby data, replacing feature engineering with data locality.

**Notation clash:* the PDF calls the local weights $w^{(i)}$, which collides with our parameter vector $w \rightarrow$ renamed $r^{(i)}$ here.

*Ordinary LR vs LWR : LWR adds the weight term to the hypothesis class

Ordinary LR: fit w minimizing $\sum_i (y^{(i)} - w^T x^{(i)})^2$, output $w^T x$ (fit once, reuse everywhere)

LWR: fit w minimizing $\sum_i r^{(i)} (y^{(i)} - w^T x^{(i)})^2$, output $w^T x$ — re-fit for every query point

$r^{(i)} (\geq 0) = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$: how much example i matters

- $x^{(i)}$ close to $x \rightarrow r^{(i)} \approx 1 \rightarrow$ try hard to fit that point
- $x^{(i)}$ far to $x \rightarrow r^{(i)} \approx 0 \rightarrow$ ignore it
- τ = bandwidth: controls how fast weight falls off with distance (small τ = very local, wiggly; large τ = approaches ordinary LR). Choosing τ = a hyperparameter problem $\rightarrow$ cross validation (9.3) applies as-is
- $r^{(i)}$ looks like a Gaussian density but isn't one — the $r^{(i)}$ are not random variables, no probabilistic meaning
- Calculating $(x^{(i)} - x)^2$:
 $(x^{(i)} - x)^T (x^{(i)} - x)$ or with Σ^{-1} inserted: $(x^{(i)} - x)^T \Sigma^{-1} (x^{(i)} - x)$

*Parametric vs Non-parametric(!=deterministic) — LWR is our first non-parametric algorithm:

Parametric: “size” of the model is fixed in advance

- Linear regression
- Neural networks

Non-parametric models: model size is a function of the training set

- KNN, Decision Trees, LWR (Cost: every prediction = one fresh weighted least-squares fit)

Logistic Regression Model

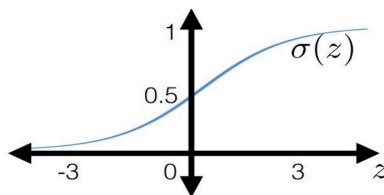
Hypothesis Class: Logistic(sigmoid) function // So-called logistic regression

$$\sigma(w^T h(x))$$

We make predictions by thresholding: e.g. 1 if $\sigma(w^T h(x)) > 0.5$, -1 if otherwise

Threshold varies with appropriateness: if false positives are more costly, set it higher than 0.5, if false negatives are more costly set it lower than 0.5

*Sigmoid function: Function going from $\mathbb{R}$ to $(0, 1)$, written as $\sigma(z) = \frac{1}{1+\exp(-z)}$



$$\sigma(0) = 0.5$$

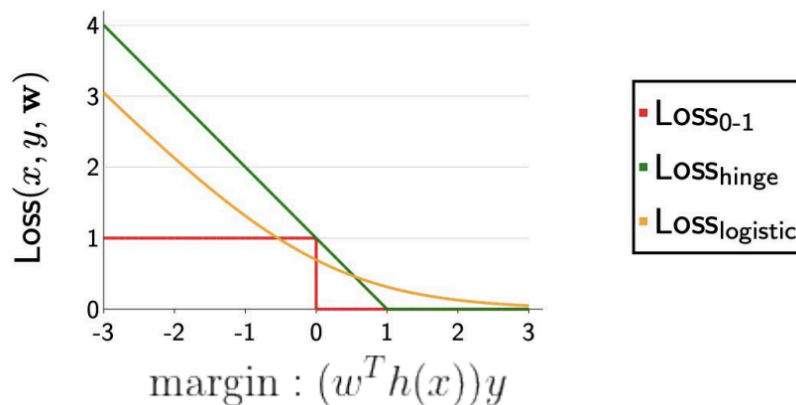
$$\sigma(-\infty) \rightarrow 0$$

$$\sigma(+\infty) \rightarrow 1$$

Loss function: Logistic Loss

$$Loss_{logistic}(x^{(i)}, y^{(i)}, w) = \log(1 + e^{\{-w^T h(x^{(i)})y^{(i)}\}})$$

$$TrainLoss_{logistic}(x^{(i)}, y^{(i)}, w) = \frac{1}{n} \sum_{i=1}^n Loss_{logistic}(x^{(i)}, y^{(i)}, w)$$



Where does logistic loss come from - probabilistic interpretation(MLE)

*We use the sigmoid function to express how likely a certain data point exists

$$P(y^{(i)} = +1 | x^{(i)}; w) = \text{Sigmoid}(x^{(i)}; w) = \frac{1}{1 + e^{-\text{Score}(x^{(i)})}} = \frac{1}{1 + e^{(\theta^T x^{(i)} + \theta_0)}}$$

$$P(y^{(i)} = +1 | x^{(i)} = x) = \sigma(\theta^T x + \theta_0)$$

$$P(y^{(i)} = -1 | x^{(i)} = x) = 1 - \sigma(\theta^T x + \theta_0)$$

$$\text{통합} \rightarrow P(y^{(i)} = y | x^{(i)} = x) = \sigma(\theta^T x + \theta_0)^{1_{\{y=+1\}}} \cdot \{1 - \sigma(\theta^T x + \theta_0)\}^{1_{\{y=-1\}}}$$

*Probability of all examples

$$P(\text{Data}) = \prod_{i=1}^n P(Y = y^{(i)} | X = x^{(i)})$$

*Find the theta that maximizes P(Data)

$$\text{argmax}_{\theta} P(\text{Data}) = \text{argmax}_{\theta} \log P(\text{Data}) = \text{argmin}_{\theta} -\log P(\text{Data})$$

$$= \text{argmin}_{\theta} -\frac{1}{n} \log P(\text{Data}) \quad // \text{average negative log likelihood}$$

$$= \text{argmin}_{\theta} -\frac{1}{n} \log \prod_{i=1}^n P(Y = y^{(i)} | X = x^{(i)}) = \text{argmin}_{\theta} -\frac{1}{n} \sum_{i=1}^n \log P(Y = y^{(i)} | X = x^{(i)})$$

$$= \text{argmin}_{\theta} -\frac{1}{n} \sum_{i=1}^n (\log \sigma(\theta^T x^{(i)} + \theta_0)^{1_{\{y^{(i)}=+1\}}} \cdot \{1 - \sigma(\theta^T x^{(i)} + \theta_0)\}^{1_{\{y^{(i)}=-1\}}})$$

$$= \text{argmin}_{\theta} -\frac{1}{n} \sum_{i=1}^n (1_{\{y^{(i)}=+1\}} \log \sigma(\theta^T x^{(i)} + \theta_0) + 1_{\{y^{(i)}=-1\}} \log \{1 - \sigma(\theta^T x^{(i)} + \theta_0)\})$$

$$= \text{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n \log(1 + e^{\{-w^T h(x^{(i)})y^{(i)}\}}) \quad // \text{generalized}$$

$$= \text{argmin}_{\theta} J(\theta)$$

*This loss is called cross-entropy loss, which is a way of measuring the difference between two probability distributions. Here, the true label distribution and the model's predicted distribution. Since the label is either 0 or 1, the "true distribution" is a spike on the correct class, which is why the loss reduces to just the negative log-probability of the correct answer. In the binary case this is called binary cross-entropy.

Optimization Algorithm: Gradient Descent

For computational convenience, we use $y \in \{0, 1\}$ instead of $y \in \{-1, +1\}$, and the loss function is now differentiable and convex to the parameters

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n (y^{(i)} (\log \sigma(\theta^T x^{(i)} + \theta_0) + (1 - y^{(i)}) \log \{1 - \sigma(\theta^T x^{(i)} + \theta_0)\}))$$

$$\text{Let } \sigma(\theta^T x^{(i)} + \theta_0) = g^{(i)},$$

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n (y^{(i)} (\log g^{(i)} + (1 - y^{(i)}) \log (1 - g^{(i)}))$$

$$\text{Use the property } \frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z)),$$

$$\nabla J(\theta) = -\frac{1}{n} \sum_{i=1}^n (y^{(i)} - g^{(i)}) x^{(i)}$$

Logistic vs. Linear Regression Gradients

Logistic Regression	$J(\theta) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log g^{(i)} + (1 - y^{(i)}) \log(1 - g^{(i)})$
	$\nabla_{\theta} J(\theta) = -\frac{1}{n} \sum_{i=1}^n (y^{(i)} - g^{(i)}) x^{(i)} = -\frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - \sigma(\theta^T x^{(i)} + \theta_0) \right) x^{(i)}$

Linear Regression	$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - g^{(i)})^2$
	$\nabla_{\theta} J(\theta) = -\frac{2}{n} \sum_{i=1}^n (y^{(i)} - g^{(i)}) x^{(i)} = -\frac{2}{n} \sum_{i=1}^n \left(y^{(i)} - (\theta^T x^{(i)} + \theta_0) \right) x^{(i)}$

Intuition: Adjust model parameters to **point in the same direction as datum**, weighted by how “wrong” the current model is (residual)

Optimization Algorithm: Newton's Method

Step 1 — Newton's method for root finding. Given $f: R \rightarrow R$, find θ with $f(\theta) = 0$. Update:

$$\theta := \theta - \frac{f(\theta)}{f'(\theta)}$$

Geometric meaning: draw the tangent line to f at the current guess, jump to where **that line** hits zero, repeat. Each iteration solves a linear approximation of the problem exactly — which is why it converges in very few steps.

Step 2 — from roots to optimization. Maxima of ℓ are where $\ell'(\theta) = 0$. So run Newton's root-finding on $f = \ell'$:

$$\theta := \theta - \frac{\ell'(\theta)}{\ell''(\theta)}$$

(Same formula for minimizing — the second derivative's sign handles the direction automatically.)

Step 3 — vector-valued θ (Newton–Raphson).

$$\theta := \theta - H^{-1} \nabla_{\theta} \ell(\theta)$$

where H is the $d \times d$ (or $(d + 1) \times (d + 1)$ with intercept) **Hessian**:

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}$$

Compare with GD: $\theta := \theta + \alpha \nabla \ell$. Newton replaces the hand-tuned scalar α with H^{-1} — instead of guessing a step size, it *divides by the curvature*, taking big steps where the function is flat and small careful steps where it curves sharply.

Tradeoff.

- **Pro:** typically far fewer iterations than (batch) GD to get very close to the optimum — no learning rate to tune.
- **Con:** each iteration must compute and invert a $d \times d$ Hessian ($\approx O(d^3)$) — same cost structure as the normal equation. Fine when d is small; prohibitive when d is large (which is why deep learning uses first-order methods).

When Newton's method is applied to maximize the logistic regression log-likelihood, the resulting method is called **Fisher scoring**.

Connections.

- GD convergence theory (pset 1(d)) said: if the max eigenvalue of the Hessian is $\beta_{\max}$, keep $\alpha < 1/\beta_{\max}$ — i.e., *shrink the step where curvature is big*. Newton automates exactly this, per-direction, by multiplying by H^{-1} .
- Normal equation vs GD (Ch 1) and Newton vs GD are the same tradeoff twice: exact/second-order methods pay $O(d^3)$ per solve, iterative/first-order methods pay per-step but scale to large d . (In fact, for the quadratic RSS objective, one Newton step from anywhere lands exactly on the normal-equation solution — Newton is the normal equation's iterative generalization to non-quadratic losses.)

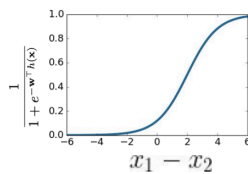
Overfitting in logistic regression

*Background knowledge: As the norm of parameters increase, the more the sigmoid becomes steep. Look at how the sigmoid graph changes along with input features below

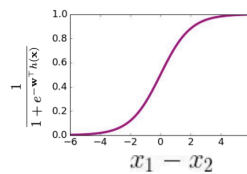
Input Features: x_0, x_1, x_2

Parameters: w_0, w_1, w_2

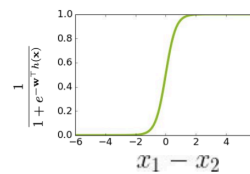
w_0	-2
w_1	+1
w_2	-1



w_0	0
w_1	+1
w_2	-1

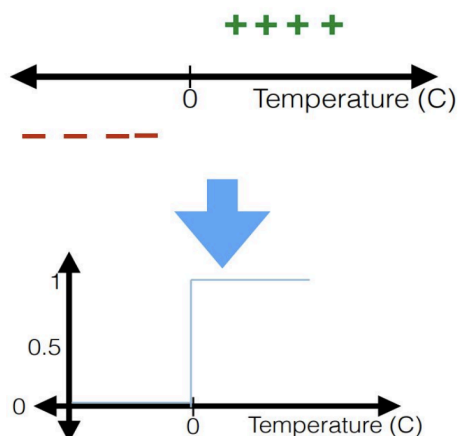


w_0	0
w_1	+3
w_2	-3

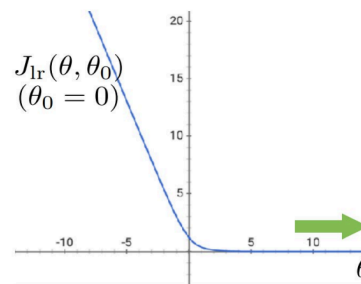


*So overfitting happens when the data is perfectly separable - since the optimization will go towards making the parameters as big as possible to minimize the training loss. So the loss function is convex, but there will be no minimum.

Perfectly separable case



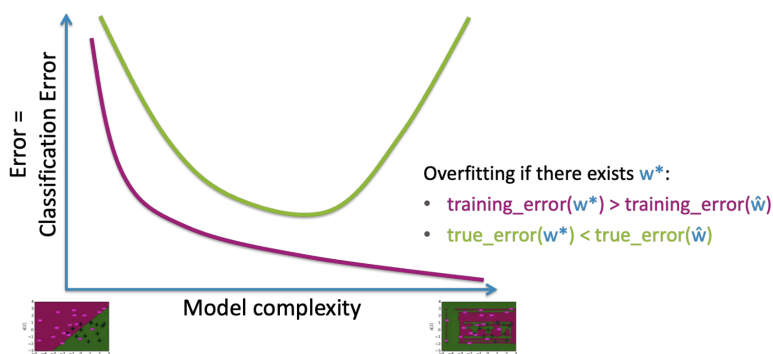
Loss is convex but there is no minimum



(If not perfectly separable, then a global, unique minimum exists)

*The rest of the overfitting and bias-variance tradeoff notions remains the same as lr

Overfitting in Classifiers



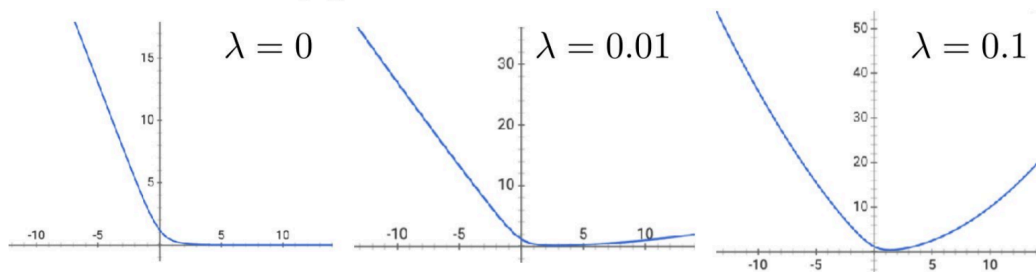
(The error plotted here is 0-1 classification error — the true target — while training still minimizes the logistic surrogate from before.)

We use regularization to mitigate this

Regularization - L2

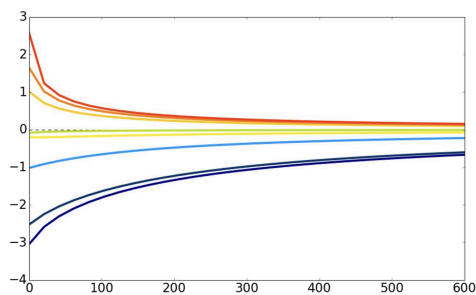
L2 regularization: penalizes being overly certain

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log g^{(i)} + (1 - y^{(i)}) \log(1 - g^{(i)}) + \lambda \|\theta\|^2$$

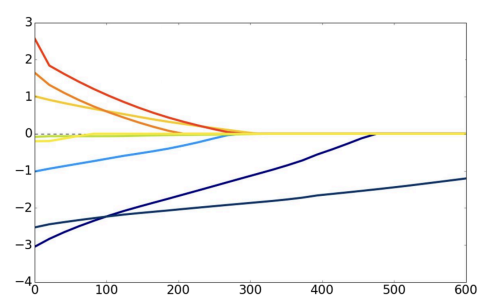


1. Global minimum now exists for perfectly separable case
2. Good idea even in non-separable cases to reduce overfitting

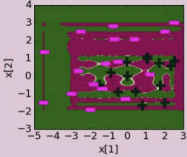
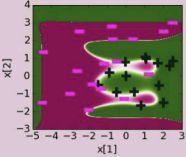
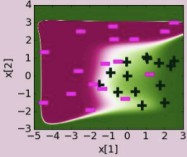
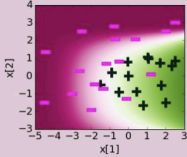
Regularization - L2 Coefficients Path



Regularization - L1 Coefficients Path



Regularization - L2 (Degree 20 Features)

Regularization	$\lambda = 0$	$\lambda = 0.00001$	$\lambda = 0.001$	$\lambda = 1$
Range of coefficients	-3170 to 3803	-8.04 to 12.14	-0.70 to 1.25	-0.13 to 0.57
Learned probabilities				

The notion of Classification

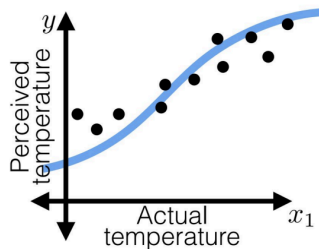
Linear Classifiers

Regression

Feature Vector: $x^{(i)} = [x_1^{(i)}, \dots, x_d^{(i)}] \in \mathbb{R}^d$

Label: $y^{(i)} \in \mathbb{R}$

Hypothesis: $h : \mathbb{R}^d \rightarrow \mathbb{R}$

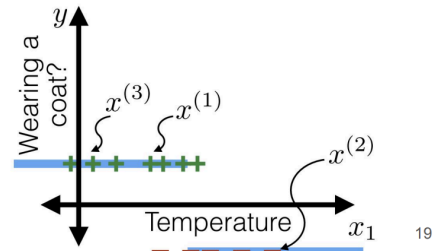


(Two-Class) Classification

Feature Vector: $x^{(i)} = [x_1^{(i)}, \dots, x_d^{(i)}] \in \mathbb{R}^d$

Label: $y^{(i)} \in \{-1, +1\}$

Hypothesis: $h : \mathbb{R}^d \rightarrow \{-1, +1\}$



A hyperplane can be described by the equation:

$$0 = w_0 + \sum_{j=1}^d w_j \cdot h_j(x)$$

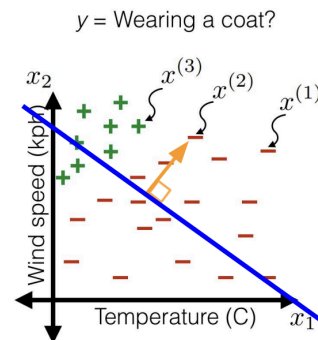
$$0 = w^T h(x)$$

$[w_1, \dots, w_d]$ is vector that is **normal** (or orthogonal) to the hyperplane, and w_0 is the **constant**.

$$\text{Score}(x) = w^T h(x)$$

Decision rule:

$$f(x; w) = \text{Sign}(\text{Score}(x)) = \begin{cases} +1, & \text{if } (\text{Score}(x)) > 0 \\ -1, & \text{otherwise} \end{cases}$$



Score: $(w^T h(x))$ - How confident we are in predicting +1

Margin: $(w^T h(x))y$ - How correct we are

Binary classification: two kinds of models

Input x , label $y \in \{0, 1\}$, prediction $\hat{y} = h(x)$.

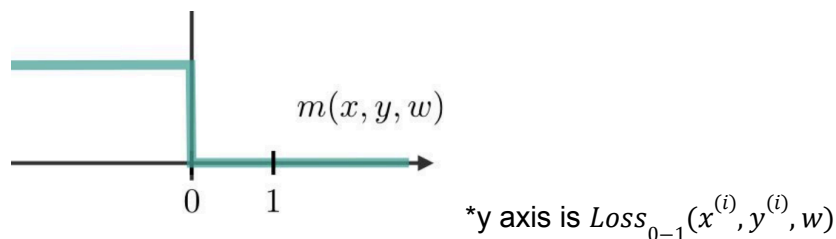
	Output	Examples
Direct-label models	a class, immediately	k-nearest neighbors, decision trees
Score-based models (our focus)	a real-valued score $s(x)$ — a margin (SVM) or a probability (logistic regression, NN)	predict positive iff the score clears a threshold τ : $\hat{y} = \mathbf{1}\{s(x) \geq \tau\}$

A score-based model is really a **whole family of classifiers** — one per threshold. A direct-label model is just the special case with the threshold baked in. Keep score and threshold **separate in your head**: the model learns the score; *you* choose τ .

	Regression	Classification
Prediction $f_w(x)$	score	$\text{sign}(\text{score})$
Relate to target y	residual (score $- y$)	margin (score y)
Loss functions	squared absolute deviation	zero-one hinge logistic
Algorithm	gradient descent	gradient descent

- Zero-One Loss

$$\text{Loss}_{0-1}(x^{(i)}, y^{(i)}, w) = 1[f_w(x^{(i)}) \neq y^{(i)}] = 1[\text{sign}(w^T h(x^{(i)})) \neq y^{(i)}]$$



Problem: there are only two modes, and gradient is zero almost everywhere

- Hinge Loss

$$\text{Loss}_{\text{hinge}}(x^{(i)}, y^{(i)}, w) = \max\{1 - w^T h(x^{(i)})y^{(i)}, 0\}$$

$$\begin{aligned} \nabla \text{Loss}_{\text{hinge}}(x^{(i)}, y^{(i)}, w) &= -h(x^{(i)})y^{(i)} && \text{if: } 1 - (w^T h(x^{(i)}))y^{(i)} > 0 \\ &= 0 && \text{if: otherwise} \end{aligned}$$

*Why these three losses?

The 0-1 loss (step function) is what we actually care about — it just counts misclassifications — but it's flat almost everywhere and has a discontinuous jump, so its gradient is zero or undefined and we can't optimize it directly. Logistic loss and hinge loss are convex, differentiable surrogates: smooth upper bounds on the 0-1 loss that we can run gradient descent on. So we minimize the surrogate (logistic loss) as a stand-in, hoping it drives the 0-1 error down too. Note the gap: the loss we train (cross-entropy / logistic) is not the metric we ultimately judge by (0-1 classification error).

Multi-Class Classification(Softmax regression / Multinomial logistic regression)

Extending from binary logistic regression, now $y \in \{1, 2, \dots, k\}$ instead of two classes. ($k \neq 2$)

Hypothesis class

$\text{Softmax}(\theta h(x))^T$ // vector with length k - one for each probability

Predict with $\text{argmax}_k \text{Softmax}(\theta h(x))^T$ or just $\text{argmax}_k \theta h(x)^T$

// softmax doesn't impact the relative order

We model $p(y|x; \theta)$ as a multinomial distribution. We assign a probability to each y class, denoting each as $\phi_1, \dots, \phi_k$, where $\phi_i = P(y = i|x; \theta)$. $\sum_{i=1}^k \phi_i = 1$.

We assign one vector per class: $\theta_1, \dots, \theta_k$, each in R^d . The natural idea is to use $\theta_1^T x, \dots, \theta_k^T x$ as the probabilities for each class. However the two problems are: (1) $\theta_j^T x$ is not necessarily in $[0, 1]$, and (2) they don't sum to 1. So we put them into a softmax function.

*Softmax

Define $\text{softmax}: R^k \rightarrow R^k$:

$$\text{softmax}(t_1, \dots, t_k) = \left[\frac{\exp(t_1)}{\sum_{j=1}^k \exp(t_j)}, \dots, \frac{\exp(t_k)}{\sum_{j=1}^k \exp(t_j)} \right]$$

The inputs $(t_1, \dots, t_k)$ are called logits. The output is always a valid probability vector (nonnegative, sums to 1). Exp makes everything positive, dividing by the sum normalizes.

Plug in $t_j = \theta_j^T x$ $\rightarrow$

$$\text{softmax}(\theta_1^T x, \dots, \theta_k^T x) = \left[\frac{\exp(\theta_1^T x)}{\sum_{j=1}^k \exp(\theta_j^T x)}, \dots, \frac{\exp(\theta_k^T x)}{\sum_{j=1}^k \exp(\theta_j^T x)} \right] = [\phi_1, \dots, \phi_k]$$

Loss function

We then calculate the loss with negative log-likelihood.

For single example $(x^{(i)}, y^{(i)})$:

$$\ell \left(\left(\theta_1^\top x^{(i)}, \dots, \theta_k^\top x^{(i)} \right), y^{(i)} \right) = -\log \phi_{y^{(i)}} = -\log \left(\frac{\exp(\theta_{y^{(i)}}^\top x^{(i)})}{\sum_{j=1}^k \exp(\theta_j^\top x^{(i)})} \right)$$

$$\text{TrainLoss} = \sum_{i=1}^n -\log \left(\frac{\exp(\theta_{y^{(i)}}^\top x^{(i)})}{\sum_{j=1}^k \exp(\theta_j^\top x^{(i)})} \right)$$

- negative log $\rightarrow$ push ϕ_y toward 1 and the loss goes to 0; let it shrink and the loss blows up.
- Each loss for a data point is a scalar - the loss only “judges” the probability assigned to the correct class t_y every data point
- This is the multi-class generalization of the cross-entropy loss we saw for logistic regression (categorical cross-entropy); for $K=2$ it reduces back to the binary case. Implementation note: cross-entropy functions differ in what they expect as input, as some take raw logits and apply (log-)softmax internally, while others take an already-softmaxed probability vector (varies by library). Don't apply softmax twice.

Optimization Algorithm: Gradient Descent

$$** \frac{\partial \ell(\theta)}{\partial \theta_j} = \frac{\partial \ell(\theta)}{\partial t_j} \cdot \frac{\partial t_j}{\partial \theta_j}$$

For the first part,

$$\frac{\partial \ell \left(\left(\theta_1^\top x^{(i)}, \dots, \theta_k^\top x^{(i)} \right), y^{(i)} \right)}{\partial (\theta_{y^{(i)}}^\top x^{(i)})} = \frac{\partial}{\partial (\theta_{y^{(i)}}^\top x^{(i)})} \left(- \log \left(\frac{\exp(\theta_{y^{(i)}}^\top x^{(i)})}{\sum_{j=1}^k \exp(\theta_j^\top x^{(i)})} \right) \right) = \phi_{y^{(i)}} - 1$$

$$\frac{\partial \ell \left(\left(\theta_1^\top x^{(i)}, \dots, \theta_k^\top x^{(i)} \right), y^{(i)} \right)}{\partial (\theta_{s \neq y^{(i)}}^\top x^{(i)})} = \frac{\partial}{\partial (\theta_{s \neq y^{(i)}}^\top x^{(i)})} \left(- \log \left(\frac{\exp(\theta_{y^{(i)}}^\top x^{(i)})}{\sum_{j=1}^k \exp(\theta_j^\top x^{(i)})} \right) \right) = \phi_{s \neq y^{(i)}}$$

$$\frac{\partial \ell \left(\left(\theta_1^\top x^{(i)}, \dots, \theta_k^\top x^{(i)} \right), y^{(i)} \right)}{\partial (\theta_1^\top x^{(i)}, \dots, \theta_k^\top x^{(i)})} = \phi - e_{y^{(i)}} \quad (\text{Vectorized}) \quad e_{y^{(i)}}: \text{one-hot vector for the correct class}$$

So gradient of loss function respect to the logit = (softmaxed probability vector) – (one-hot answer).

For the second part,

$$t_j = \theta_j^\top x, \text{ so } \frac{\partial t_j}{\partial \theta_j} = x.$$

$$\text{Result: } \frac{\partial \ell(\theta)}{\partial \theta_j} = \frac{\partial \ell(\theta)}{\partial t_j} \cdot \frac{\partial t_j}{\partial \theta_j} = \sum_{i=1}^n \left(\phi_j^{(i)} - 1_{\{y^{(i)} = j\}} \right) (x^{(i)})^T \quad // \text{ k x d matrix}$$

This is the k -class version of the binary intuition (residual $\times$ datum): if the model already puts probability 1 on the right class, $\phi - e_y = 0$ and nothing moves. Same shape as every gradient so far: (error) $\times$ (input). Can run (stochastic) gradient descent with this.

$$\theta \leftarrow \theta - \alpha \frac{\partial \ell(\theta)}{\partial \theta_j} = \theta - \alpha \sum_{i=1}^n \left(\phi_j^{(i)} - 1_{\{y^{(i)} = j\}} \right) (x^{(i)})^T$$

Connections

- $k = 2$ recovers binary logistic regression: *softmax* of two logits = $\sigma(t_1 - t_2)$, so binary sigmoid is a special case of softmax.
- The vectorized gradient $\phi - e_y$ is reused as-is in deep learning — a neural net classifier is “some network producing logits t , then ℓ_{ce}^n ”; backprop starts from exactly $\phi - e_y$.
- The modular decomposition here — loss as a function of the logits, then chain rule through $\partial t / \partial \theta$ — is the same pattern as for in binary ($\partial \ell / \partial t = \sigma(t) - y$).

Generalized Linear Models (GLMs)

Motivating observation - So far we've built two models, and if we state them side by side they look suspiciously parallel:

	Distribution assumed	How the distribution parameter connects to x, θ
Linear regression	$y x; \theta \sim N(\mu, \sigma^2)$	$\mu = \theta^\top x$
Logistic regression	$y x; \theta \sim \text{Bernoulli}(\phi)$	$\phi = \sigma(\theta^\top x)$

Only the distribution differs and the rest of the recipe (pick a distribution, tie its parameter to $\theta^\top x$, run MLE) is identical. Actually, both are special cases of one family - GLMs. We can also generate new models (Poisson regression, softmax, etc.) from the same machinery.

The Exponential Family

Definition: A class of distributions is in the exponential family if it can be written as
 $p(y; \eta) = b(y) \exp(\eta^T T(y) - a(\eta))$ // η is the parameter of the child distribution

- $\eta(\theta)$ = natural parameter (could be a vector)
- $T(y)$ = sufficient statistic (could be a vector (usually $T(y) = y$))
- $a(\eta)$ = log partition function; $e^{-a(\eta)}$ is constant making p integrate to 1.
 - * $a'(\eta) = E(T(y)) = \mu$ (mean of the distribution)
 - * $a''(\eta) = \text{Var}[T(y)] \rightarrow$ since variance is nonnegative, a is convex
 - * In general, $\nabla_{\eta} a(\eta) = E[T(y)]$, $\nabla_{\eta}^2 a(\eta) = \text{Cov}[T(y)]$
- $b(y)$ = base measure
- * $T(y)$, $b(y)$ should only depend on y and $\eta(\theta)$, $a(\eta)$ should only depend on $\eta(\theta)$

A fixed choice of T, a, b defines a family and varying η moves you around within that family. Some exponential family examples: Bernoulli (Binary), Multinomial=Categorical (Multiple classes), Gaussian (Real valued), Poisson (Counts), Gamma / Exponential (Positive reals), Dirichlet (Distributions)

Example 1 — Bernoulli is in the exponential family.

$$p(y = 1; \phi) = \phi$$

$$p(y = 0; \phi) = 1 - \phi$$

so

$$p(y; \phi) = \phi^y (1 - \phi)^{1-y} = \exp(\log(\phi^y (1 - \phi)^{1-y})) = \exp(y \log \phi + (1 - y) \log(1 - \phi)) \\ = \exp\left(\left(\log\left(\frac{\phi}{1-\phi}\right)\right)y - (-\log(1 - \phi))\right)$$

Matching to the template:

$$\eta = \log\left(\frac{\phi}{1-\phi}\right), \quad T(y) = y, \quad a(\eta) = -\log(1 - \phi) = \log(1 + e^{\eta}), \quad b(y) = 1$$

$$\left(\log\left(\frac{\phi}{1-\phi}\right)\right)^{-1} \text{ is sigmoid}$$

Example 2 — Gaussian is in the exponential family.

When we derived linear regression, σ^2 had no effect on the final θ or $h_{\theta}(x)$, so set $\sigma^2 = 1$ for simplicity:

$$p(y; \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}|y - \mu|^2\right) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right) \cdot \exp\left(\mu y - \frac{1}{2}\mu^2\right)$$

so

$$\eta = \mu, \quad T(y) = y, \quad a(\eta) = \frac{\mu^2}{2} = \frac{\eta^2}{2}, \quad b(y) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{y^2}{2}\right)$$

*Footnote: if you keep σ^2 as a variable, the Gaussian is still in the family, but $\eta \in \mathbb{R}^2$ depends on both μ and σ . Alternatively use the generalized form

$$p(y; \eta, \tau) = b(a, \tau) \exp\left(\left(\eta^T T(y) - a(\eta)\right)/c(\tau)\right), \text{ where } \tau \text{ is the dispersion parameter and } c(\tau) = \sigma^2 \text{ for the Gaussian. Not needed for this example.}$$

Generalized Linear Models

y	x
$y^{(1)}$	$- \quad x^{(1)} \quad -$
$y^{(2)}$	$- \quad x^{(2)} \quad -$
$\dots$	$\dots$
$y^{(n)}$	$- \quad x^{(n)} \quad -$

-> Based on this data, given new x , how do we predict y ?

Since the world is inherently noisy, we don't say : "If we know x , we know what y exactly is." Rather, we say : "If we know x , we know the probability of what each y value will be - a.k.a what the probability distribution of y is. We then can look at that distribution and choose an appropriate y value, saying "Based on x , this is our best guess of y ." (In GLMs, that appropriate value will always be the mean of that distribution - $E[y | x]$)

So we need to find a generalized probabilistic relationship between x and y for prediction. How do we learn that relationship? We learn it from the given data, assuming the given data was generated under the same underlying probabilistic relationship.

In GLMs we say the probabilistic relationship(= probability distribution of y given x) is one from the 'exponential family'. The exponential family takes in one canonical parameter (denoted as $\eta(\theta)$) that determines how the distribution looks like. Since different $x^{(i)}$ s is the only thing that makes the probability distribution of $y^{(i)}$ vary, and same $\eta(\theta)$ s produce the same probability distribution, $x^{(i)}$ s should be related to the canonical parameter. We generalize the relationship between all the $x^{(i)}$ s and the canonical parameters with w :
 $\eta(\theta)^{(i)} = w^T x^{(i)}$.

Now we have the general probability distribution $p(y^{(i)} | \eta(\theta)^{(i)})$ where $\eta^{(i)} = w^T x^{(i)}$. So now it's clear having the right θ is crucial - since θ is what determines how the probability distribution of y given x looks like, having the right θ lets us correctly predict y with the mean of the distribution.

So how do we choose the right θ ? Assuming that the given data is independent and identically distributed, the θ that maximizes the likelihood of seeing the given data altogether is the best θ .

Since the data is IID, the probability of seeing the given data altogether is: $\prod_{i=1}^n p(y_i | \eta_i)$.

So we should find $\operatorname{argmax}_{\theta} \prod_{i=1}^n p(y_i | \eta_i)$.

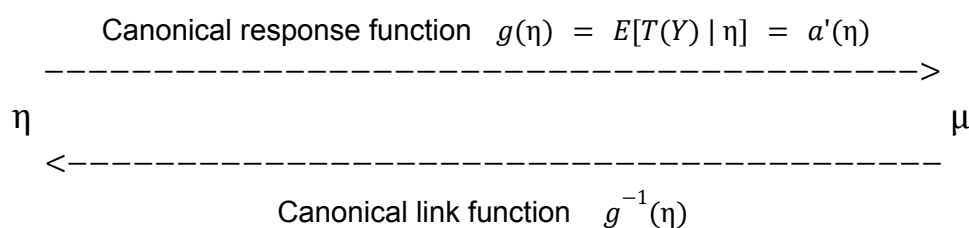
$$\begin{aligned} \operatorname{argmax}_{\theta} \prod_{i=1}^n p(y_i | \eta_i) &= \operatorname{argmax}_{\theta} \log \prod_{i=1}^n p(y_i | \eta_i) = \operatorname{argmax}_{\theta} \sum_{i=1}^n \log p(y_i | \eta_i) \\ &= \operatorname{argmin}_{\theta} \sum_{i=1}^n -\log p(y_i | \eta_i) = \operatorname{argmin}_{\theta} - \sum_{i=1}^n \log [b(y_i) \cdot \exp((\eta_i)^T T(y_i) - a(\eta_i))] \\ &= \operatorname{argmin}_{\theta} - \sum_{i=1}^n \log b(y_i) + (\eta_i)^T T(y_i) - a(\eta_i) \\ &= \operatorname{argmin}_{\theta} \sum_{i=1}^n a(\eta_i) - (\eta_i)^T T(y_i) \end{aligned}$$

So the loss function is: $a(\eta_i) - (\eta_i)^T T(y_i)$

$$\begin{aligned} \nabla_{\theta} [a(\eta_i) - (\eta_i)^T T(y_i)] &= \sum_{i=1}^n a'(\eta_i) \cdot x_i - T(y_i) \cdot x_i = \sum_{i=1}^n (\mu - T(y_i)) \cdot x_i \\ &= \sum_{i=1}^n (\hat{y} - T(y_i)) \cdot x_i \quad // \hat{y} \text{ denotes our prediction, and we predict with the mean} \end{aligned}$$

So the gradient of the loss function is: $\sum_{i=1}^n (\hat{y}_i - T(y_i)) \cdot x_i$

*Terminology: Canonical response function, Canonical Link function



Handwritten notes with examples coming next